# Othello-Spieler – Kurzbeschreibung der Lösung

Modul: Intelligente Systeme

Betreuer: Prof. Dr. J. Priemer

Dominik Wendland (`spieler.wendo`)

23. Juli 2026

## 1 Überblick

Der Spieler implementiert die vorgegebene Schnittstelle `OthelloSpieler`. Die Zugberechnung erfolgt mit Hilfe eines Spielbaums durch eine *Minimax-Suche*[1], die in der *Negamax*-Formulierung umgesetzt ist. Das *Alpha-Beta-Pruning* ist dabei als *NegaScout*[2] realisiert. Als *iterative Tiefensuche* unter Zeitkontrolle vertieft sich die Suche schrittweise (Tiefe $1, 2, 3, \dots$), bis das Zeitbudget des Zuges erschöpft ist, und liefert den besten Zug der letzten vollständig durchsuchten Tiefe. Zur effizienten Beschneidung des Spielbaums werden die Züge an jedem Knoten heuristisch sortiert – an jedem inneren Knoten nach statischer Feldbewertung und *Killerzügen*, ergänzt um die Prinzipalvariante aus der iterativen Tiefensuche. Die Positionsbewertung ist phasenabhängig und gewichtet im Mittel- und Endspiel unterschiedliche Merkmale. Die folgenden Abschnitte gehen auf die einzelnen Aspekte der Lösung ein.

## 2 Architektur

Der Code ist nach dem Single-Responsibility-Prinzip modular in fünf Klassen mit folgenden Aufgaben gegliedert:

- `Spieler` – Adapter zur Wettkampf-Umgebung, übersetzt `Farbe` und `Zug` in die interne Darstellung und zurück, pflegt das eigene Spielbrett und behandelt Sonderfälle (Passen, ungültiger Gegnerzug → `ZugException`).
- `Board` – Spielregeln und Spielstand (Zuggenerierung, Ausführen inkl. Umdrehen, Endstellungserkennung).
- `MiniMax` – Suchverfahren (NegaScout, Zugsortierung, iterative Tiefensuche, Zeitabbruch).
- `Evaluate` – Positionsbewertung.
- `TimeManager` – Aufteilung der Gesamtbedenkzeit auf die einzelnen Züge.

## 3 Brettdarstellung

Das $8 \times 8$-Brett ist ein flaches `int`-Array der Länge 64 und zeilenweise mit Index = Zeile $\cdot\, 8 +$ Spalte indiziert (Zeile 0, Spalte 0 ist das Feld oben links). Jede Zelle trägt einen der Werte `EMPTY`, `BLACK` oder `WHITE` und bildet so die Framework-Farben `Farbe.SCHWARZ` und `Farbe.WEISS` ab. Die Suche folgt dem *copy-make*-Prinzip. Für jeden zu prüfenden Zug wird über `copy()` eine Kopie des Boards angelegt und der Zug nur auf dieser Kopie ausgeführt, sodass die Ausgangsstellung unberührt bleibt. Eine solche Kopie ist ein Array aus 64 Werten und wird als Ganzes per `System.arraycopy` dupliziert. Da die Alpha-Beta-Suche je Zug sehr viele Knoten expandiert und an jedem Knoten eine Kopie erzeugt, muss das Duplizieren einer Stellung billig sein, was ein flaches Primitiv-Array leistet.

## 4 Zugberechnung

Bei jedem Aufruf von `berechneZug` wird zunächst der letzte Zug des Gegners auf dem eigenen Brett synchronisiert (`replayOpponentMove()`), anschließend über `TimeManager` das Zeitbudget für diesen Zug bestimmt und die Suche gestartet.

**Suchverfahren (NegaScout).** Die Suche ist als Negamax formuliert. Beide Spieler maximieren aus eigener Sicht, und bei jedem Halbzug wird das Vorzeichen der Kindbewertung negiert. Darauf setzt NegaScout auf. Der laut Zugsortierung vielversprechendste Zug wird mit vollem Suchfenster untersucht, jeder weitere zunächst nur mit einem Nullfenster als billige Probe „ist dieser Zug überhaupt besser?“. Erst wenn die Probe dies andeutet (*fail high*), wird der Zug mit vollem Fenster nachgesucht. Bei guter

[1] S. J. Russell und P. Norvig, Künstliche Intelligenz: ein moderner Ansatz, 4. Aktualisierte Auflage, Pearson, 2023.

[2] A. Reinefeld, Spielbaum-Suchverfahren, Bd. 200. in Informatik-Fachberichte, Springer Berlin Heidelberg, 1989

Zugsortierung expandiert NegaScout dadurch deutlich weniger Knoten als einfaches Alpha-Beta, denn die meisten Geschwisterzüge werden schon vom billigen Nullfenster widerlegt und nur die wenigen tatsächlich besseren Züge kosten eine volle Nachsuche.

**Zugsortierung.** Alpha-Beta bzw. NegaScout beschneiden den Spielbaum nur dann effizient, wenn der beste Zug zuerst untersucht wird. An jedem inneren Knoten wird die Zugreihenfolge daher heuristisch bestimmt:

- **Statische Feldbewertung** – Vorsortierung nach einer festen Gewichtstabelle (Ecken sehr hoch, ecknahe „X“/„C“-Felder negativ).
- **Killerzüge** – pro Suchtiefe werden bis zu zwei Züge gemerkt, die zuletzt einen Beta-Schnitt ausgelöst haben, und an Geschwisterknoten zuerst probiert. Ein Zug, der einen Geschwisterknoten widerlegt, widerlegt häufig auch die übrigen. Auf einer festen Stellung mit exakter Knotenzählung (`NodeBenchmark`) senkte diese Maßnahme die Knotenzahl im Mittelspiel bei Tiefe 10 um rund 37 % (2,27 auf 1,43 Mio.).

Über die Iterationen hinweg tritt an der Wurzel die Prinzipalvariante hinzu: Der beste Zug der vorherigen, flacheren Suche wird in der nächsten Iteration zuerst probiert.

**Iterative Tiefensuche und Zeitmanagement.** Statt sofort mit fester Tiefe zu suchen, wird nacheinander mit Tiefe $1, 2, 3, \ldots$ gesucht. Das hat zwei Vorteile: Es liegt jederzeit ein vollständiges Ergebnis vor (wichtig für die Zeitkontrolle), und jede flachere Iteration verbessert die Zugsortierung der nächsten. `TimeManager` weist jedem Zug ein Budget aus der verbleibenden Bedenkzeit geteilt durch die geschätzte Restzugzahl (etwa die Hälfte der freien Felder) zu, vermindert um eine Sicherheitsreserve von 10 %. Läuft das Budget während einer Iteration ab, bricht die Suche über eine `TimeUpException` sofort ab und liefert den besten Zug der letzten vollständig abgeschlossenen Tiefe. Die Tiefe ist dabei nur eine Obergrenze, real begrenzt die Zeit. Im Endspiel mit wenigen freien Feldern wird die Partie faktisch bis zum Ende ausgerechnet.

## 5 Positionsbewertung

Die Bewertungsfunktion `evaluate()` liefert einen Wert aus Sicht der ziehenden Seite und ist antisymmetrisch ($\text{bewerte}(b, \text{seite}) = -\text{bewerte}(b, \text{gegner})$), was die Negamax-Formulierung der Suche voraussetzt. Sie ist phasenabhängig und wechselt bei 50 Steinen auf dem Brett von einer heuristischen Mischung zu einer zählenden Bewertung.

- **Mittelspiel** (unter 50 Steinen): gewichtete Summe aus *Eckenkontrolle*, *Mobilität* (normierte Differenz der Zugmöglichkeiten) und *Positionswert* (Feldtabelle). Die Ecken tragen das größte Gewicht, denn ein besetzter Eckstein kann nie wieder umgedreht werden und ist damit dauerhaft sicher. Die reine Steinzahl bleibt bewusst unberücksichtigt, weil ein früher Steinvorsprung in Othello meist nachteilig ist, da er die eigene Mobilität einengt und dem Gegner Möglichkeiten zum Umdrehen liefert.
- **Endspiel** (ab 50 Steinen): Die Mischung wird durch die *Steindifferenz* ersetzt, weiterhin mit starker Gewichtung der Ecken. Nun entscheidet die endgültige Steinzahl über Sieg oder Niederlage, sodass Mobilität und Feldtabelle ihren Vorhersagewert verlieren und stattdessen direkt gezählt wird.

## 6 Fazit

Der Spieler verbindet eine NegaScout-Suche mit iterativer Tiefensuche, dreistufiger Zugsortierung und einer phasenabhängigen Bewertung. Leistungsänderungen wurden an exakten Knotenzahlen überprüft und nur bei nachgewiesenem Nutzen übernommen, das Verhalten ist durch 48 JUnit-Tests abgesichert. Im Wettbewerb belegte der Spieler dennoch nur den vierten Platz – weniger, als die Optimierung der Suche erwarten ließ. Die folgende Einordnung ist eine Hypothese, keine Diagnose. Die Suchoptimierung gewann über die Geschwindigkeit zwar mehr Tiefe, doch dürfte die von Hand gewichtete Bewertung nicht ausreichend abgestimmt gewesen sein. Die vielversprechendsten Hebel wären entsprechend beispielsweise eine sorgfältiger abgestimmte oder gelernte Bewertung und ein früher exakt gelöstes Endspiel. Welcher davon tatsächlich entscheidet, könnte erst eine Analyse der Turnierpartien zeigen.